

VINUNIVERSITY

Data Pipeline Engineering

AICB-P2T2 · Ngày 17 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

“Agent hỗ trợ khách hàng của bạn bắt đầu trả lời sai. Model không đổi một dòng nào. Sai ở bảng feature nó đọc: một job chạy lệch múi giờ, một schema đổi ở upstream, một partition thiếu. Ai là người dựng — và sửa — đường ống đó?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Bronze & Ingestion: CDC, connector, định dạng
2. Transform: dbt, SQLMesh, incremental
3. Engine: DuckDB, Polars, Spark, Trino, Ray
4. Streaming: Kafka, Flink, và họ hàng
5. Orchestration: Airflow, Dagster, Temporal

Mục tiêu

- Đọc được bản đồ công nghệ data engineering 2026 và biết mỗi thứ nằm ở tầng nào
- Chọn công cụ theo *ràng buộc*, không theo thời trang
- Viết transform incremental không nhân đôi dữ liệu
- Phân biệt event time vs processing time

Ngày 10 Đã Học Gì — Hôm Nay Khác Gì

Ngày 10 — *hôm nay không nhắc lại*

- Data pipeline là gì, gồm những chặng nào
- ETL vs ELT — chọn cái nào cho AI
- Batch vs Streaming ở mức khái niệm
- 6 dimensions of data quality
- "Airflow là orchestrator, dbt là transform"

Ngày 17 — nghề data engineer

- CDC log-based hoạt động ra sao, khác polling chỗ nào
- Parquet vs Avro vs Arrow — dùng ở đâu
- dbt vs SQLMesh; Airflow vs Dagster vs Temporal
- Năm engine tính toán, chọn theo kích thước dữ liệu
- Flink vs Spark Streaming: event-at-a-time vs micro-batch

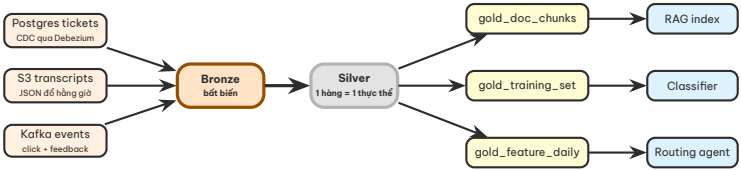
Lưu ý: Ngày 10 hỏi *"dùng gì"*. Hôm nay hỏi *"nó chạy thế nào bên trong, và vì sao chọn nó chứ không phải cái kia"*.

Bản Đồ Công Nghệ Data Engineering 2026



Bản đồ này là "mục lục" của buổi hôm nay. Mỗi tầng ta sẽ mở ra một lần — xem nó làm gì, và chọn cái nào.

Bài Toán Xuyên Suốt: Nền Tảng AI Hỗ Trợ Khách Hàng



Mọi công cụ hôm nay sẽ được gắn vào đúng một chỗ trên sơ đồ này. Ba nhánh Gold, ba consumer AI, ba định nghĩa "sạch" khác nhau.

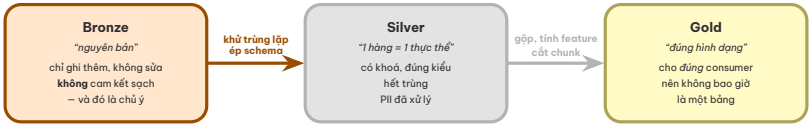
Bronze & Ingestion: Data Vào Kho Bằng Đường Nào

CDC, connector, và định dạng — ba quyết định bạn khó đổi về sau

01

Medallion: Ba Mức Cam Kết Về Dữ Liệu

Medallion không phải ba cái tên bảng. Nó là **ba mức cam kết** — mỗi mức nói rõ nó hứa gì, và *không* hứa gì.



Lưu ý: Bronze trả lời *chuyện gì đã xảy ra*. Silver trả lời *hiện trạng là gì*. Gold trả lời *consumer này cần gì*. Ba câu hỏi khác nhau — nên phải là ba bảng khác nhau. (*Lưu bằng format nào* — Delta, Iceberg — là Ngày 18.)

Bronze — Cam Kết: “Đây Là Bản Gốc. Không Ai Được Sửa.”

Cam kết là gì

Bất cứ ai đọc Bronze đều được tin *một* điều: dữ liệu ở đây **y hệt lúc nó rời khỏi hệ thống nguồn**. Không ai chỉnh sửa, không ai làm sạch, không ai “sửa nhẹ cho gọn”.

Chỉ `INSERT`. Không `UPDATE`. Không `DELETE`.

“Nguyên bản” đến mức nào

- `priority` là `'HIGH '` thừa dấu cách? **Lưu nguyên.**
- Ticket bị gửi trùng 3 lần? **Lưu cả 3.**
- Có trường lạ không ai biết là gì? **Lưu luôn.**

```
-- Bronze mang theo dau vet duong di
CREATE TABLE bronze_tickets (
  _payload      JSON,      -- nguyen ban
  _source       VARCHAR,   -- nguon nao
  _op           CHAR(1),   -- c / u / d
  _ingested_at  TIMESTAMP, -- nap luc nao
  _batch_id     VARCHAR,   -- lan chay nao
  _kafka_offset BIGINT     -- de replay
);
```

Lưu ý: Bronze **không** cam kết dữ liệu sạch — đây là *chủ ý thiết kế*, không phải sự cẩu thả.

Vì Sao Phải Giữ Nguyên Bản: Ba Tình Huống Rất Thật

1 · Logic Silver viết sai

Phát hiện sau 2 tháng.

Bronze còn nguyên → xoá Silver, chạy lại, **xong trong 20 phút**.

Bronze đã bị "làm sạch" → phải đi xin lại từ Postgres. Mà Postgres chỉ có *trạng thái hôm nay*, không có lịch sử 2 tháng. **Không cứu được.**

2 · Retrain model

Model cần học từ dữ liệu *như nó vốn có tại thời điểm đó*.

Nếu ai đó đã "chuẩn hoá" Bronze theo quy tắc mới của tháng này, bạn đang huấn luyện model trên **một quá khứ đã bị viết lại**.

3 · Audit

Sáu tháng sau có người hỏi:

"Model từ chối yêu cầu của khách hàng này dựa trên dữ liệu gì?"

Chỉ Bronze mới trả lời được — và chỉ khi nó chưa từng bị sửa.

Lưu ý: Bronze là cuốn sổ ghi chép, không phải bản báo cáo. Sổ ghi chép mà tẩy xoá được thì không còn giá trị làm bằng chứng.

Silver — Cam Kết: “Một Hàng = Một Thực Thể”

Bronze — mỗi hàng = một lần thay đổi

_op	ticket_id	priority	status	_ingested_at
c	T-91	low	open	08-10 09:00
u	T-91	high	open	08-14 11:30
u	T-91	high	closed	08-16 15:00

Silver — mỗi hàng = một ticket

ticket_id	priority	status	updated_at
T-91	high	closed	08-16 15:00

Lưu ý: Không có bước này, **mọi người dùng ở tầng sau đều đếm sai**. Analyst hỏi “tháng này bao nhiêu ticket?” → đếm trên Bronze ra 3, sự thật là 1.

Lưu ý: Model học từ Bronze thấy T-91 xuất hiện 3 lần → ngầm hiểu *“loại ticket này quan trọng gấp 3”* — trong khi nó chỉ đơn giản là bị sửa nhiều lần hơn.

Ngược lại: một “Silver” chỉ là `SELECT * FROM bronze WHERE user_id IS NOT NULL` thì *không cam kết thêm điều gì* — đó là **layer thừa**, xoá đi pipeline nhanh hơn và rẻ hơn.

Silver — Có Khoá, Đúng Kiểu, PII Đã Xử Lý

VINUNIVERSITY

Có khoá — và vì sao nó quyết định tất cả

Khoá = cột nhận diện duy nhất một thực thể (`ticket_id`): **không NULL**, **không trùng**.

Có khoá → `MERGE` được: "có T-91 thì cập nhật, chưa có thì thêm". Chạy 1 hay 10 lần vẫn đúng 1 hàng.

Không khoá → chỉ `INSERT`. Airflow retry lúc 3 giờ sáng → nhân đôi; lần nữa → nhân ba. *Và không gì báo lỗi.*

Đúng kiểu

`VARCHAR` thì mãi mãi là `VARCHAR`. Không ép kiểu: upstream đổi `priority` số sang chuỗi, pipeline chạy êm, **3 tuần sau** mới có người thấy dashboard ra số lạ.

PII đã xử lý

Silver là **ranh giới cuối cùng**. Qua khỏi đây dữ liệu toả ra Gold, feature store, vector DB, file CSV ai đó export — không gom lại được.

Lưu ý: Silver là layer đầu tiên dám nói về chất lượng — đọc Silver là được quyền tin, không cần kiểm lại.

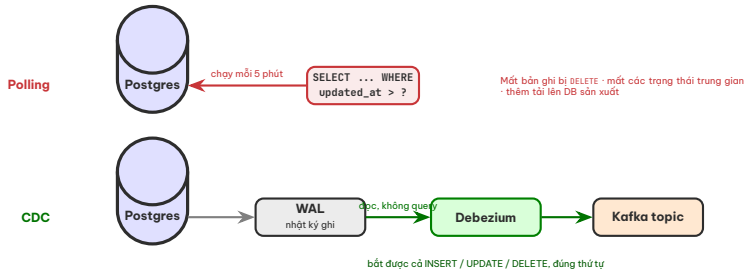
Gold — Cam Kết: “Đúng Hình Dạng Cho Đúng Người Dùng”

Gold không cố đúng cho *mọi* người. Nó cam kết đúng cho **một** consumer — đúng đến mức dùng được ngay, không phải xử lý gì thêm.

Bảng / Consumer	1 hàng là	Cam kết sống còn	Nếu sai thì sao
`gold_doc_chunks` RAG index	một chunk văn bản	chunk ổn định + ghim version model embedding	đổi model mà không ghim → index trộn hai hệ toạ độ vector, kết quả nhiễu mà không ai hiểu vì sao
`gold_training_set` Classifier	một ticket đã gán nhãn	snapshot bất biến, có version	tháng sau model tụt điểm, không trả lời được “nó đã học từ tập nào”
`gold_feature_daily` Routing agent	một user × một ngày	point-in-time đúng, không rò rỉ tương lai	feature 12-08 lỡ dùng nhãn sinh ra 15-08 → đẹp lúc test, sập lúc production

Lưu ý: Ba consumer hiểu chữ “sạch” theo ba kiểu *mâu thuẫn trực tiếp* với nhau — nên Gold không bao giờ là một bảng.

CDC Log-Based: Debezium Đọc Thẳng WAL



Lưu ý: Postgres dùng *logical replication* trên WAL, MySQL dùng *binlog*. Debezium giả làm một replica — nên gần như không thêm tải truy vấn. Cái giá: phải bật replication slot, và slot không được đọc sẽ làm đầy đĩa của DB gốc.

Chọn Công Cụ Ingestion

Công cụ	Mô hình	Chọn khi	Trả giá
Debezium	CDC log-based, chạy trên Kafka Connect	cần *mọi* thay đổi, kể cả xoá; cần thứ tự	tự vận hành, phải trông replication slot
Airbyte	mã nguồn mở, rất nhiều connector	nhiều nguồn SaaS lặt vặt, ngân sách hẹp	connector chất lượng không đều
Fivetran	SaaS quản lý trọn gói	muốn hết việc vận hành, chấp nhận trả tiền	giá theo số dòng, khó dự đoán
Kafka Connect	framework source/sink	đã có Kafka, cần đổ vào/ra S3, ES, JDBC	phải hiểu Connect cluster
Tự viết	script Python	nguồn quá lạ, không ai hỗ trợ	bạn vừa nhận thêm một hệ thống phải bảo trì

Trong ví dụ hôm nay: **Debezium** cho Postgres tickets · **Kafka Connect S3 Sink** cho transcripts · producer sẵn có cho events.

Định Dạng: Avro Trên Đường, Parquet Khi Lưu, Arrow Trong RAM

Định dạng	Cấu trúc	Mạnh ở	Dùng tại chặng nào
JSON	văn bản, không schema	người đọc được, ai cũng parse được	chỉ ở rìa hệ thống; đừng lưu lâu dài
Avro	theo *hàng*, có schema	ghi nhanh, tiến hoá schema là tính năng gốc	**trên đường truyền** — message Kafka
Parquet	theo *cột*, nén mạnh	chỉ đọc cột cần; đẩy được điều kiện lọc xuống file	**khi lưu** — mọi bảng trong lake
Arrow	theo cột, *trong bộ nhớ*	trao đổi giữa engine không cần chuyển đổi	**trong RAM** — DuckDB ↔ Polars ↔ Spark

Vì sao Arrow quan trọng

Đọc Parquet bằng DuckDB rồi đưa sang Polars xử lý tiếp: không tốn một lần chuyển đổi nào, vì cả hai cùng nói Arrow. Trước Arrow, mỗi lần đổi công cụ là một lần serialize lại.

Lưu ý: Đừng lưu Bronze bằng JSON nén gzip "cho tiện". Sáu tháng sau, một truy vấn lọc một cột sẽ phải giải nén và parse toàn bộ lịch sử — và hoá đơn sẽ nhắc bạn nhớ ngày hôm nay.

02

Transform: Biến Đổi Dữ Liệu Như Viết Phần Mềm

dbt, SQLMesh — và những kỹ thuật khiến transform chạy lại được

dbt: Model, `ref()`, Materialization, Contract

VINUNIVERSITY

Ba tầng model

1. `staging` — 1:1 với nguồn, làm sạch nhẹ
2. `intermediate` — join, logic nghiệp vụ
3. `marts` — bảng Gold cuối cùng

Materialization

`view` khi phát triển → `table` khi chạy thật → `incremental` khi bảng lớn.

`{{ ref() }}` tự sinh đồ thị phụ thuộc — dbt biết thứ tự chạy mà bạn không cần khai báo.

```yaml
# models/silver/schema.yml
models:
  - name: silver_tickets
    config:
      contract: {enforced: true}  # sai kieu = fail
    columns:
      - name: ticket_id
        data_type: varchar
        constraints: [{type: not_null}]
        tests: [unique]
      - name: label
        data_type: varchar
        tests:
          - accepted_values:
              values: ['bug','billing','other']
```

Contract kiểm lúc *build* — không phải lúc tình cờ phát hiện dashboard sai.

dbt Incremental: Ba Dòng Quyết Định Tất Cả

```
{{ config(
     materialized   = 'incremental',
     unique_key     = 'ticket_id',
     incremental_strategy = 'merge'
) }}

select ticket_id, user_id, event_date,
       label, _ingested_at
from {{ ref('silver_tickets') }}

{% if is_incremental() %}
  -- lookback 3 ngay: bat ban ghi ve muon
  where event_date >=
        (select max(event_date) from {{ this }})
        - interval 3 day
{% endif %}
```

Ba dòng đó

- `unique_key` — thiếu là dữ liệu nhân đôi mỗi lần chạy lại
- `incremental_strategy` — `merge` = upsert; `delete+insert` = ghi đè cả phân vùng
- `is_incremental()` — thiếu thì mỗi lần chạy quét lại toàn bộ lịch sử

Lưu ý: `--full-refresh` chỉ an toàn khi Bronze còn đủ lịch sử. Nếu Bronze đã bị xoá theo TTL, nó *dựng lại một bảng thiếu data* — và không có gì báo cho bạn biết.

SQLMesh: Đối Thủ Đáng Chú Ý Của dbt

	dbt	SQLMesh
Hiểu SQL đến đâu	xử lý SQL như *văn bản* + Jinja	**phân tích cú pháp** thành cây → lineage tới từng *cột*
Khi bạn sửa model	bạn tự đoán ảnh hưởng	tự phân loại *breaking* hay không, và chỉ backfill phần cần
Môi trường dev	thường phải build lại toàn bộ schema riêng	*virtual environment* — tạo view, gần như không tốn tính toán
Hệ sinh thái	rất lớn; dễ tuyển người	nhỏ hơn nhiều, tài liệu ít hơn

Chọn thế nào

dbt nếu team mới, cần cộng đồng và tuyển dụng dễ. **SQLMesh** nếu bạn đã đau vì backfill nhầm và vì môi trường dev quá đắt.

Lưu ý: Điểm chung quan trọng hơn khác biệt: cả hai đều bắt bạn coi transform là **code có version, có test, có review** — thay vì một tập script SQL trong thư mục dùng chung.

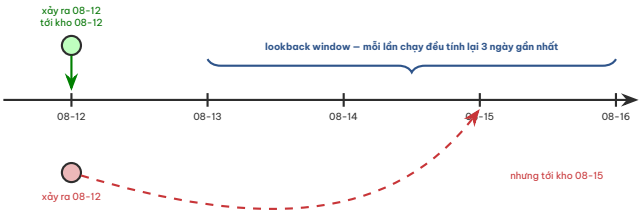
Bốn Cách Viết Idempotent

Idempotent — Chạy một lần hay N lần đều cho *cùng một trạng thái cuối*. Cần nó vì mọi hệ thống đều là **at-least-once**: Airflow retry, người bấm "Clear Task", backfill chồng lịch, consumer restart.

Kỹ thuật	Cách làm	Hợp khi	Cái giá
Overwrite partition	`DELETE` ngày X rồi `INSERT` lại	có cột thời gian rõ ràng	rẻ nhất, đơn giản nhất
MERGE / upsert	khớp theo khoá, có thì cập nhật	bản ghi bị sửa về sau (CDC)	cần khoá tin cậy
Dedup on read	`row_number()`, giữ bản mới nhất	không sửa được tầng ghi	trả giá *mỗi lần đọc*
Content hash	`md5(payload)` làm khoá	nguồn không có khoá ổn định	hash đổi khi format đổi

Mặc định: overwrite partition cho bảng theo ngày · MERGE cho bảng thực thể. Hai cái này phủ gần hết trường hợp thực tế.

Data Về Muộn: Nhãn Của Quá Khứ Có Thể Đổi



Vì sao data về muộn

App offline rồi sync · producer retry · batch upstream chạy chậm · người dùng sửa lại bản ghi cũ

Lưu ý: Người dùng bấm *không hài lòng* sau 3 ngày → nhãn của ticket 08-12 đổi. **Đừng sửa snapshot training cũ** — tạo version mới.

Đặt lookback bằng P99 của `(_ingested_at - event_time)` — đo từ Bronze, đừng đoán.

Giữ Lịch Sử: Ticket Bị Sửa Thì Sao?

VINUNIVERSITY

Overwrite — mất quá khứ

Ticket `T-91` đổi `priority` từ `low` sang `high` vào 08-14.

Bảng chỉ còn **một hàng**: `high`.

Hỏi: *lúc ticket được tạo, độ ưu tiên là gì?* — không trả lời được.

Training set gán nhãn theo trạng thái *hôm nay* → model học được thông tin mà lúc dự đoán thật nó không hề có.

SCD Type 2 — giữ lịch sử

Mỗi lần đổi sinh một hàng mới:

`valid_from` · `valid_to` · `is_current`

`T-91 | low | 08-10 → 08-14`

`T-91 | high | 08-14 → NULL`

Join theo **point-in-time**: lấy trạng thái *tại thời điểm ticket được tạo*.

Sai chỗ này chính là **training-serving skew** — Ngày 19 đi sâu hơn.

Kiểm Thử Dữ Liệu: Bốn Công Cụ, Chọn Cái Nào

Công cụ	Kiểm ở đâu	Mạnh ở	Hợp với
dbt tests	chạy SQL trong warehouse	đi liền với model, không thêm hạ tầng	bảng Silver / Gold
Pandera	trong tiến trình Python	kiểm schema DataFrame (pandas, Polars, PySpark)	bước xử lý bằng Python
Pydantic	từng bản ghi, lúc chạy	bắt sai kiểu ngay tại biên hệ thống	payload API, message Kafka
Great Expectations / Soda	khung riêng, có báo cáo	bộ kỳ vọng dùng chung nhiều team, tài liệu tự sinh	tổ chức lớn

Lưu ý: Quy tắc đặt chốt kiểm: **ngay sau extract** (bắt lỗi nguồn) → **sau transform** (bắt lỗi logic) → **trước khi vào training** (chốt cuối). Hỏng sớm thì rẻ. Ngày 27 đi sâu vào suite nâng cao và cảnh báo tự động.

Engine Tính Toán: Năm Lựa Chọn

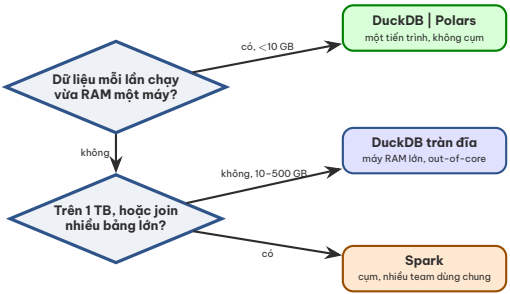
Không có engine tốt nhất — có engine đúng kích thước dữ liệu

Năm Engine Bạn Sẽ Gặp

Engine	Mô hình thực thi	Mạnh nhất ở	Trả giá
DuckDB	trong tiến trình, một máy, vector hoá	phân tích tới vài trăm GB; đọc thẳng Parquet trên S3	không mở rộng ngang được
Polars	DataFrame Rust, thực thi trễ (lazy)	thay thế pandas; ETL bằng Python nhanh hơn nhiều lần	hệ sinh thái còn trẻ
Spark	phân tán, JVM	TB đến PB; join nhiều bảng lớn; hệ sinh thái khổng lồ	vận hành cụm, khởi động chậm
Trino	MPP, truy vấn liên kho	hỏi xuyên S3 + Postgres + Kafka *mà không di chuyển data*	không phải nơi chạy transform theo lịch
Ray Data	phân tán, thuần Python	**batch inference / embedding trên GPU**	non hơn cho SQL analytics

Trong ví dụ hôm nay: **DuckDB** dựng Silver/Gold · **Ray Data** embed 10 triệu chunk cho RAG · **Trino** khi cần đối chiếu nhanh lake với Postgres sản xuất.

Cây Quyết Định — Và Cái Bẫy "Distributed By Default"



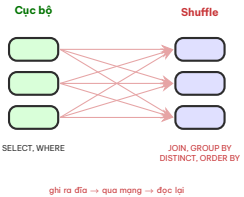
Ngoài trục này:

Trino khi câu hỏi trải trên nhiều kho khác nhau.

Ray Data khi việc nặng là gọi model chứ không phải SQL.

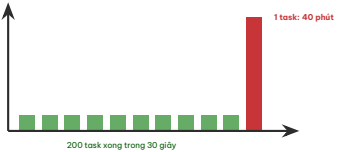
Lưu ý: Một cụm Spark 10 node xử lý 8 GB thường *chậm hơn* DuckDB trên một máy — gần như toàn bộ thời gian trôi vào lập lịch và shuffle, không phải tính toán. Bắt đầu từ một máy; chỉ lên cụm khi đã đo và thấy chật.

Scan Rẻ, Shuffle Đắt — Và Skew Còn Đắt Hơn



Bỏ shuffle

Bảng nhỏ (<100 MB) → **broadcast join**: gửi nguyên bản tới mọi node, join thành cục bộ.



Lưu ý: Data skew. Một khách hàng doanh nghiệp chiếm 40% ticket → partition theo `customer_id` dồn hết vào *một* task. Thủ phạm phổ biến thứ hai: `user_id IS NULL`.

Nhìn ra bằng *phân bố* thời gian task, không phải trung bình. Sửa bằng **salting**: gộp theo `(customer_id, ngẫu nhiên 0–15)` rồi gộp lần hai.

Small-File Problem & Chiến Lược Phân Vùng

Lưu ý: 50.000 file × 2 MB. Mỗi file là một lần liệt kê, mở, đọc footer. Engine dành phần lớn thời gian *lập kế hoạch* chứ không đọc data. Pipeline chậm dần theo tháng mà không ai đổi dòng code nào.

Cách xử lý

- Nhắm kích thước file **128 MB – 1 GB**
- Chạy compaction định kỳ
- Giảm số task ghi ra ở cuối pipeline

Phân vùng: chọn cột nào

- Partition theo cột mà bạn **hay lọc** — gần như luôn là `event_date`
- **Đừng** partition theo `user_id`: cardinality cao → quay lại đúng bài toán small-file
- Nhắm vài trăm đến vài nghìn phân vùng, không phải vài triệu

Predicate pushdown

Lọc trên *đúng cột phân vùng* thì engine bỏ qua cả thư mục — không mở file nào. Lọc cột khác thì phải đọc hết rồi mới loại.

Đo Trước Khi Tối Ưu — Và Cái Bẫy Chi Phí Embedding

```sql
-- Loc sai cot: quet toan bo
EXPLAIN ANALYZE
SELECT count(*) FROM gold_feature_daily
WHERE customer_name = 'ACME';
--  rows scanned: 412,000,000
--  elapsed: 38.2s

-- Loc dung cot phan vung
EXPLAIN ANALYZE
SELECT count(*) FROM gold_feature_daily
WHERE event_date = DATE '2026-08-15'
  AND customer_name = 'ACME';
--  rows scanned: 1,340,000
--  elapsed: 0.4s
```

Quy tắc

Đọc `rows scanned` trước, đừng đọc thời gian. Thời gian đổi theo tải máy; số hàng phải quét thì không nói dối.

Lưu ý: Embedding 10 triệu chunk bằng Ray Data. Nếu `gold_doc_chunks` không idempotent, mỗi lần chạy lại là *embed lại toàn bộ* — tiền thật, mỗi lần.

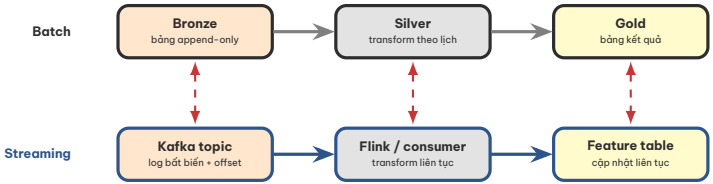
Khoá đúng: `hash(text) + embedding_model_version`. Đổi model thì re-embed toàn bộ — **có chủ đích**, và không trộn lẫn hai version trong cùng một index.

04

Streaming: Chính Medallion Đó, Trên Trục Thời Gian

Broker, stream processor, và bài toán thời gian sự kiện

Kafka Topic = Bronze Có Hạn Sử Dụng



Ánh xạ một-một

Topic bất biến, chỉ ghi thêm — đúng luật Bronze. `offset` là vị trí đọc riêng của từng consumer group. **Replay** = đọc lại từ offset cũ = chính là "chạy lại pipeline".

Lưu ý: `retention` quyết định bạn replay được bao xa. **Retention 24 giờ mà lookback 3 ngày** → không backfill được, và bạn phát hiện ra đúng lúc cần nó nhất.

Chọn Broker: Năm Lựa Chọn

Broker	Đặc điểm	Chọn khi / trả giá
Kafka	chuẩn de-facto, hệ sinh thái lớn nhất	mặc định an toàn · vận hành nặng, JVM cần tinh chỉnh
Redpanda	viết bằng C++, một binary, tương thích Kafka API	muốn độ trễ thấp và ít việc vận hành · cộng đồng nhỏ hơn
Pulsar	tách broker và lưu trữ, đa tenant, tiered storage sẵn có	một cụm phục vụ nhiều team · nhiều thành phần phải hiểu
Kinesis	AWS quản lý hoàn toàn	đã ở AWS, muốn bật là chạy · khoá vào AWS, giới hạn theo shard
WarpStream	dùng object storage làm log, không đĩa cục bộ	chi phí quan trọng hơn độ trễ · độ trễ hàng trăm ms

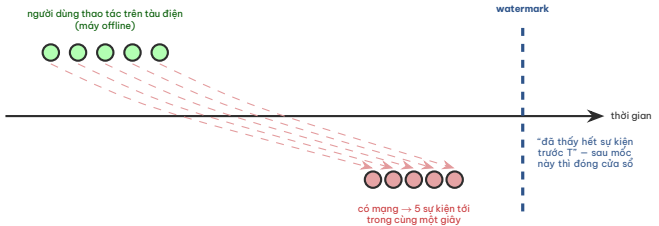
Lưu ý: Cả năm đều nói *gần như* cùng một API. Điều thật sự khoá bạn lại không phải broker — mà là **schema**, **partition key** và những consumer đã viết. Hãy cẩn thận với ba thứ đó hơn là với lựa chọn broker.

Chọn Stream Processor: Ba Trường Phái

	Flink	Spark Structured Streaming	Kafka Streams
Mô hình	từng sự kiện một, đúng nghĩa streaming	micro-batch (gom theo lô nhỏ)	thư viện nhúng trong app Java
Độ trễ	mili giây	giây	mili giây
State	phong phú, checkpoint bài bản	có, nhưng đơn giản hơn	RocksDB cục bộ
Chọn khi	event-time và state là trung tâm	team đã dùng Spark, muốn *một* API cho cả batch lẫn stream	chỉ biến đổi Kafka → Kafka
Trả giá	dốc nhất về vận hành	không đạt được mili giây	không phải cụm riêng, khó scale độc lập

Cả ba đều có API SQL (**Flink SQL**, **Spark SQL**, **ksqlDB**) — bạn không bắt buộc phải viết Java để bắt đầu.

Event Time vs Processing Time



Lưu ý: Nếu cửa sổ tính theo **processing time**, cả 2 giờ hoạt động dồn vào một phút. Feature "số sự kiện mỗi phút" vọt lên giả tạo → agent định tuyến tưởng khách hàng đang gặp sự cố nghiêm trọng. Cửa sổ phải tính theo **event time**.

Allowed lateness là biên độ tha thứ sau watermark. Sự kiện tới muộn hơn: bỏ, hoặc đẩy sang nhánh phụ — *đừng lặng lẽ vứt đi.*

Schema Registry: Data Contract Cho Stream

Chế độ	Cho phép đổi gì	Ai nâng cấp trước	Dùng khi
BACKWARD	xoá field; thêm field có giá trị mặc định	**consumer**	mặc định — nhiều consumer, ít producer
FORWARD	thêm field; xoá field có giá trị mặc định	**producer**	producer nâng cấp thường xuyên
FULL	chỉ những thay đổi an toàn cả hai chiều	bất kỳ bên nào	topic quan trọng, nhiều bên phụ thuộc

Nối lại với phần 1

Đây chính là **data contract** của Medallion — nhưng do máy chủ ép buộc *ngay lúc producer ghi*, chứ không phải lúc pipeline chạy và phát hiện ra đã muộn.

Lưu ý: Đổi schema của event = đổi feature = **model đang chạy nhận input khác đi** mà không ai deploy lại model. Với topic nuôi feature online, chọn **FULL** và chấp nhận đổi chậm hơn.

Khi Nào KHÔNG Cần Streaming

Nhu cầu trong hệ AI	Độ trễ thật sự cần	Giải pháp rẻ nhất
Cập nhật RAG index từ tài liệu nội bộ	giờ	batch mỗi giờ
Retrain classifier phân loại ticket	ngày	batch chạy đêm
Feature định tuyến agent (số ticket 7 ngày)	phút	micro-batch 5 phút
Chặn giao dịch gian lận	mili giây	**streaming thật**

Chi phí thật của streaming

State store phải quản lý · checkpoint và khôi phục · backpressure khi consumer chậm · trực 24/7 · debug khó hơn batch vì không có "chạy lại cho tôi xem".

Lưu ý: Micro-batch 5 phút giải quyết phần lớn nhu cầu "real-time" với một phần nhỏ chi phí vận hành. Chọn streaming vì **yêu cầu độ trễ**, không vì nó nghe hiện đại hơn trong buổi review kiến trúc.

Orchestration & Tổng Kết

05

Ai bấm nút chạy, và chuyện gì xảy ra khi cần chạy lại

Bốn Trường Phái Điều Phối

Công cụ	Trung tâm là	Mạnh ở	Trả giá
Airflow	*task* — DAG các bước	hệ sinh thái provider lớn nhất; bản 3 thêm lịch theo sự kiện và versioning DAG	cấu hình nhiều, DAG dễ phình
Dagster	*asset* — bảng, model, file	khai báo "tài sản dữ liệu"; lineage và observability có sẵn	phải nghĩ lại theo asset
Prefect	*flow* Python	flow động, ít nghi thức, viết như code thường	ít mặc định riêng cho data
Temporal	*durable execution*	workflow nhiều ngày, agent nhiều bước; retry nằm trong runtime	không phải scheduler cho data

Lưu ý: Khác biệt lớn nhất là *đơn vị tư duy*: Airflow hỏi "chạy những bước nào", Dagster hỏi "những bảng nào phải tồn tại và còn tươi". Với AI agent nhiều bước, **Temporal** ngày càng được chọn — dù nó không thuộc thế giới ETL.

Chạy Lại & Backfill An Toàn

Bốn quy tắc

- `catchup=False` — deploy DAG mới với mặc định `True` sẽ chạy *mọi lịch đã lỡ* cùng lúc
- `max_active_runs=1` — backfill không chồng lên lần chạy hằng ngày
- Backfill dùng **đúng code path** với daily. Script riêng = kết quả riêng
- Ghi có phân vùng → backfill ngày 08-12 không đụng ngày 08-13

Lưu ý: Kịch bản có thật: deploy DAG training mới lúc 5 giờ chiều, `start_date` để 30 ngày trước, `catchup` để mặc định. Airflow lập tức xếp 30 lần chạy — và cụm GPU nhận 30 job cùng lúc.

Bài kiểm tra cuối cùng

Chạy lại một ngày cũ ba lần liên tiếp, ghi checksum bảng Gold sau mỗi lần. **Ba con số phải giống hệt nhau** — đây cũng là tiêu chí chấm Lab 17.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Mỗi tầng có công cụ của nó, và mỗi công cụ có một ràng buộc rõ ràng: **Debezium** khi cần cả bản ghi bị xoá · **Avro** trên đường truyền, **Parquet** khi lưu, **Arrow** trong bộ nhớ · **dbt** cho hệ sinh thái, **SQLMesh** cho lineage tới từng cột.

2. Không có engine tốt nhất — có engine **đúng kích thước**. DuckDB và Polars trên một máy đi xa hơn bạn tưởng; Spark khi thật sự cần; Trino để hỏi xuyên kho; Ray Data khi việc nặng là gọi model chứ không phải SQL.

3. Streaming là bài toán **thời gian sự kiện**, không phải "batch nhanh hơn" — chọn nó vì yêu cầu độ trễ. Và dù dùng công cụ nào, transform vẫn phải chạy lại được: ba lần, cùng một checksum.

LAB #17

Mục tiêu: Dựng đường ống cho nền tảng AI hỗ trợ khách hàng bằng chính stack hôm nay — Debezium/Kafka → Bronze Parquet → dbt trên DuckDB → ba bảng Gold — rồi sửa ba lỗi đã cài sẵn trong repo.

Deliverable: Repo chạy được + **ba lần chạy cho ra checksum giống hệt nhau** + báo cáo một trang giải thích từng lỗi, từng lựa chọn công cụ.

Thời gian: 2,5 giờ

Bài tiếp theo

Ngày 18
Data Lakehouse Architecture

"Hôm nay ta chọn công cụ cho từng tầng. Ngày mai: định dạng bảng nào cho phép ACID, time travel và schema evolution ngay trên object storage"

Bài tập về nhà

- Hoàn thành Lab 17 — nộp cả ba checksum
- Đọc trước: Delta Lake transaction log, Apache Iceberg spec
- Đối chiếu: stack ở nơi bạn làm việc đang thiếu tầng nào trên bản đồ?

Hỏi & Đáp

Trên bản đồ công nghệ hôm nay, dự án của bạn đang đứng ở đâu — và tầng nào đang là điểm yếu nhất?



Cảm ơn!

AICB-P2T2 · Ngày 17

Data Pipeline Engineering

`lms.vinuni.edu.vn` · Slide & template trên LMS